12. Block Service 구현 분석

개요

Block Service는 Lighthouse Validator Client의 핵심 컴포넌트 중 하나로, 검증자가 블록 제안자로 선정되었을 때 블록을 생성하고 서명하여 네트워크에 제출하는 역할을 담당합니다. 이 서비스는 Duties Service로부터 블록 제안 알림을 받아 실제 블록 생성 과정을 수행하며, 슬래싱 보호와 doppelganger 보호 메커니즘을 통해 안전한 블록 제안을 보장합니다.

Block Service 아키텍처

핵심 구조체

BlockService 구조체

```
// validator_client/validator_services/src/block_service.rs
pub struct BlockService<S, T> {
inner: Arc<Inner<S, T>>,
}

pub struct Inner<S, T> {
validator_store: Arc<S>,
slot_clock: Arc<T>,
pub beacon_nodes: Arc<BeaconNodeFallback<T>>,
pub proposer_nodes: Option<Arc<BeaconNodeFallback<T>>>,
```

```
executor: TaskExecutor,
chain_spec: Arc<ChainSpec>,
graffiti: Option<Graffiti>,
graffiti_file: Option<GraffitiFile>,
}
```

주요 필드 설명:

- `validator_store` : 검증자 키 관리 및 서명 기능 제공
- `slot_clock` : 현재 슬롯 시간 추적
- `beacon_nodes` : 일반적인 beacon node 연결 관리
- `proposer_nodes` : 블록 제안 전용 beacon node (선택적)
- `executor` : 비동기 작업 실행을 위한 태스크 실행자
- `chain_spec` : 체인 사양 정보
- `graffiti` : 블록에 포함할 graffiti 메시지

BlockServiceBuilder 패턴

```
pub struct BlockServiceBuilder<S, T> {
validator_store: Option<Arc<S>>,
slot_clock: Option<Arc<T>>,
beacon_nodes: Option<Arc<BeaconNodeFallback<T>>>,
proposer_nodes: Option<Arc<BeaconNodeFallback<T>>>,
executor: Option<TaskExecutor>,
chain_spec: Option<Arc<ChainSpec>,
```

```
graffiti: Option<Graffiti>,

graffiti_file: Option<GraffitiFile>,

}



impl<S: ValidatorStore, T: SlotClock + 'static> BlockServiceBuilder<S, T> {

pub  fn  build(self) -> Result<BlockService<S, T>, String> {

Ok(BlockService {

inner: Arc::new(Inner {

validator_store: self.validator_store

.ok_or("Cannot build BlockService without validator_store")?,

slot_clock: self.slot_clock

.ok_or("Cannot build BlockService without slot_clock")?,

beacon_nodes: self.beacon_nodes

.ok_or("Cannot build BlockService without beacon_node")?,

executor: self.executor

.ok_or("Cannot build BlockService without executor")?,

chain_spec: self.chain_spec

.ok_or("Cannot build BlockService without chain_spec")?,

proposer_nodes: self.proposer_nodes,

graffiti: self.graffiti,

graffiti_file: self.graffiti_file,

}),

})
```

```
}
}
```

서비스 초기화 및 시작

서비스 시작 과정

```
impl<S: ValidatorStore + 'static, T: SlotClock + 'static> BlockService<S, T> {
pub fn start_update_service(
self,
mut notification_rx: mpsc::Receiver<BlockServiceNotification>,
) -> Result<(), String> {
info!("Block production service started");

let executor = self.inner.executor.clone();

executor.spawn(
async move {
while let Some(notif) = notification_rx.recv().await {
self.do_update(notif).await.ok();
}
debug!("Block service shutting down");
},
"block_service",
```

```
);

Ok(())
}
}
```

동작 원리:

1. `notification_rx` 채널을 통해 Duties Service로부터 블록 제안 알림 수신
2. 각 알림에 대해 `do_update()` 메서드를 비동기적으로 실행
3. 태스크 실행자를 통해 백그라운드에서 지속적으로 실행

BlockServiceNotification 구조체

```
pub struct BlockServiceNotification {
pub slot: Slot,
pub block_proposers: Vec<PublicKeyBytes>,
}
```

블록 제안 과정 분석

1. 블록 제안 알림 처리 (do_update)

```
async fn do_update(&self, notification: BlockServiceNotification) -> Result<()
let _timer = validator_metrics::start_timer_vec(
```

```
&validator_metrics::BLOCK_SERVICE_TIMES,
&[validator_metrics::FULL_UPDATE],
);

let  slot = self.slot_clock.now().ok_or_else(move || {
crit!("Duties manager failed to read slot clock");
})?;

// 슬롯 만료 검사
if  notification.slot != slot {
warn!(
current_slot = slot.as_u64(),
notification_slot = notification.slot.as_u64(),
info = "Your machine could be overloaded",
"Skipping block production for expired slot"
);
return Ok(());
}

// 제네시스 슬롯 검사
if  slot == self.chain_spec.genesis_slot {
debug!(
proposers = format!("{:?}", notification.block_proposers),
```

```
"Not producing block at genesis slot"
);
return Ok(());
}

let proposers = notification.block_proposers;

// 각 제안자에 대해 병렬로 블록 제안 실행
for validator_pubkey in proposers {
let builder_boost_factor = self
.validator_store
.determine_builder_boost_factor(&validator_pubkey);
let service = self.clone();
self.inner.executor.spawn(
async move {
let result = service
.publish_block(slot, validator_pubkey, builder_boost_factor)
.await;

match result {
Ok(_) => {}
Err(BlockError::Recoverable(e)) | Err(BlockError::Irrecoverable(e)) => {
error!(
```

```
error = ?e,
block_slot = ?slot,
info = "block v3 proposal failed",
"Error whilst producing block"
);
}
}
},
"block service",
)
}
Ok(())
}
```

주요 검증 단계:

1. **슬롯 동기화 검사**: 현재 슬롯과 알림 슬롯이 일치하는지 확인
2. **제네시스 슬롯 검사**: 제네시스 슬롯에서는 블록 제안하지 않음
3. **병렬 처리**: 여러 검증자가 동시에 제안자인 경우 병렬로 처리

2. 블록 생성 및 제출 (publish_block)

```
async fn publish_block(
self,
slot: Slot,
```

```
validator_pubkey: PublicKeyBytes,

builder_boost_factor: Option<u64>,

) -> Result<(), BlockError> {

let  _timer = validator_metrics::start_timer_vec(

&validator_metrics::BLOCK_SERVICE_TIMES,

&[validator_metrics::BEACON_BLOCK],

);



// 1. RANDAO reveal 서명 생성

let  randao_reveal = match  self

.validator_store

.randao_reveal(validator_pubkey, slot.epoch(S::E::slots_per_epoch()))

.await

{

Ok(signature) => signature.into(),

Err(ValidatorStoreError::UnknownPubkey(pubkey)) => {

warn!(

info = "a validator may have recently been removed from this VC",

?pubkey,

?slot,

"Missing pubkey for block randao"

);

return Ok(());
```

```
}
Err(e) => {
return Err(BlockError::Recoverable(format!(
"Unable to produce randao reveal signature: {:?}",
e
)))
}
};


// 2. Graffiti 결정
let  graffiti = determine_graffiti(
&validator_pubkey,
self.graffiti_file.clone(),
self.validator_store.graffiti(&validator_pubkey),
self.graffiti,
);


let  proposer_index = self.validator_store.validator_index(&validator_pubkey);
let  proposer_fallback = ProposerFallback {
beacon_nodes: self.beacon_nodes.clone(),
proposer_nodes: self.proposer_nodes.clone(),
};
```

```
info!(slot = slot.as_u64(), "Requesting unsigned block");


// 3. Beacon Node로부터 미서명 블록 요청
let  unsigned_block = proposer_fallback
.request_proposers_last(|beacon_node| async  move {
let  _get_timer = validator_metrics::start_timer_vec(
&validator_metrics::BLOCK_SERVICE_TIMES,
&[validator_metrics::BEACON_BLOCK_HTTP_GET],
);
Self::get_validator_block(
&beacon_node,
slot,
&randao_reveal,
graffiti,
proposer_index,
builder_boost_factor,
)
.await
.map_err(|e| {
BlockError::Recoverable(format!(
"Error from beacon node when producing block: {:?}",
e
))
```

```
})
})
.await?;

// 4. 블록 서명 및 제출
self.sign_and_publish_block(
proposer_fallback,
slot,
graffiti,
&validator_pubkey,
unsigned_block,
)
.await?;

Ok(())
}
```

블록 생성 단계:

1. **RANDAO Reveal 생성**: 해당 에포크에 대한 RANDAO reveal 서명 생성
2. **Graffiti 결정**: 블록에 포함할 graffiti 메시지 결정
3. **미서명 블록 요청**: Beacon Node로부터 미서명 블록 템플릿 요청
4. **블록 서명 및 제출**: 블록에 서명하고 네트워크에 제출

3. 미서명 블록 요청 (get_validator_block)

```
async fn get_validator_block(
beacon_node: &BeaconNodeHttpClient,
slot: Slot,
randao_reveal_ref: &SignatureBytes,
graffiti: Option<Graffiti>,
proposer_index: Option<u64>,
builder_boost_factor: Option<u64>,
) -> Result<UnsignedBlock<S::E>, BlockError> {
let (block_response, _) = beacon_node
.get_validator_blocks_v3::<S::E>(
slot,
randao_reveal_ref,
graffiti.as_ref(),
builder_boost_factor,
)
.await
.map_err(|e| {
BlockError::Recoverable(format!(
"Error from beacon node when producing block: {:?}",
e
))
})?;
```

```
let (block_proposer, unsigned_block) = match  block_response.data {
eth2::types::ProduceBlockV3Response::Full(block) => {
(block.block().proposer_index(), UnsignedBlock::Full(block))
}
eth2::types::ProduceBlockV3Response::Blinded(block) => {
(block.proposer_index(), UnsignedBlock::Blinded(block))
}
};

info!(slot = slot.as_u64(), "Received unsigned block");
// 제안자 인덱스 검증
if  proposer_index != Some(block_proposer) {
return Err(BlockError::Recoverable(
"Proposer index does not match block proposer. Beacon chain re-orged".to_string(
));
}

Ok(unsigned_block)
}
```

블록 요청 과정:

1. **API 호출**: `/eth/v1/validator/blocks/{slot}` 엔드포인트 호출

2. **블록 타입 처리**: Full 블록 또는 Blinded 블록 처리

3. **제안자 검증**: 요청한 검증자와 블록의 제안자가 일치하는지 확인

블록 서명 과정 분석

1. 블록 서명 및 제출 (sign_and_publish_block)

```
async fn sign_and_publish_block(
&self,
proposer_fallback: ProposerFallback<T>,
slot: Slot,
graffiti: Option<Graffiti>,
validator_pubkey: &PublicKeyBytes,
unsigned_block: UnsignedBlock<S::E>,
) -> Result<(), BlockError> {
let signing_timer = validator_metrics::start_timer(&validator_metrics::BLOCK_SI

// 1. 블록 서명
let res = self
.validator_store
.sign_block(*validator_pubkey, unsigned_block, slot)
.await;

let signed_block = match res {
```

```
Ok(block) => block,
Err(ValidatorStoreError::UnknownPubkey(pubkey)) => {
warn!(
info = "a validator may have recently been removed from this VC",
?pubkey,
?slot,
"Missing pubkey for block"
);
return Ok(());
}
Err(e) => {
return Err(BlockError::Recoverable(format!(
"Unable to sign block: {:?}",
e
)))
}
};

let  signing_time_ms =
Duration::from_secs_f64(signing_timer.map_or(0.0, |t| t.stop_and_record())).as_m

info!(
slot = slot.as_u64(),
```

```
signing_time_ms = signing_time_ms,
"Publishing signed block"
);

// 2. 서명된 블록 제출
proposer_fallback
.request_proposers_first(|beacon_node| async {
self.publish_signed_block_contents(&signed_block, beacon_node)
.await
})
.await?;

let  metadata = BlockMetadata::from(&signed_block);
info!(
block_type = ?metadata.block_type,
deposits = metadata.num_deposits,
attestations = metadata.num_attestations,
graffiti = ?graffiti.map(|g| g.as_utf8_lossy()),
slot = metadata.slot.as_u64(),
"Successfully published block"
);
Ok(())
```

```
}
```

2. Validator Store에서의 블록 서명

```
// validator_client/lighthouse_validator_store/src/lib.rs

async fn sign_block(

&self,

validator_pubkey: PublicKeyBytes,

block: UnsignedBlock<E>,

current_slot: Slot,

) -> Result<SignedBlock<E>, Error> {

match block {

UnsignedBlock::Full(block) => {

let (block, blobs) = block.deconstruct();

self.sign_abstract_block(validator_pubkey, block, current_slot)

.await

.map(|block| {

SignedBlock::Full(PublishBlockRequest::new(Arc::new(block), blobs))

})

}

UnsignedBlock::Blinded(block) => self

.sign_abstract_block(validator_pubkey, block, current_slot)

.await
```

```
.map(|block| SignedBlock::Blinded(Arc::new(block))),
}
}
```

3. 추상 블록 서명 (sign_abstract_block)

```
async fn sign_abstract_block<Payload: AbstractExecPayload<E>>(
&self,
validator_pubkey: PublicKeyBytes,
block: BeaconBlock<E, Payload>,
current_slot: Slot,
) -> Result<SignedBeaconBlock<E, Payload>, Error> {
// 1. 슬롯 검증
if block.slot() > current_slot {
warn!(
block_slot = block.slot().as_u64(),
current_slot = current_slot.as_u64(),
"Not signing block with slot greater than current slot"
);
return Err(Error::GreaterThanCurrentSlot {
slot: block.slot(),
current_slot,
});
```

```
}

let signing_epoch = block.epoch();
let signing_context = self.signing_context(Domain::BeaconProposer, signing_epoc
let domain_hash = signing_context.domain_hash(&self.spec);

// 2. Doppelganger 보호 확인
let signing_method = self.doppelganger_checked_signing_method(validator_pubkey)

// 3. 슬래싱 보호 검사
let slashing_status = if signing_method
.requires_local_slashing_protection(self.enable_web3signer_slashing_protection)
{
self.slashing_protection.check_and_insert_block_proposal(
&validator_pubkey,
&block.block_header(),
domain_hash,
)
} else {
Ok(Safe::Valid)
};

match slashing_status {
```

```
// 4. 안전한 서명 수행
Ok(Safe::Valid) => {
validator_metrics::inc_counter_vec(
&validator_metrics::SIGNED_BLOCKS_TOTAL,
&[validator_metrics::SUCCESS],
);


let  signature = signing_method
.get_signature(
SignableMessage::BeaconBlock(&block),
signing_context,
&self.spec,
&self.task_executor,
)
.await?;
Ok(SignedBeaconBlock::from_block(block, signature))
}
Ok(Safe::SameData) => {
warn!("Skipping signing of previously signed block");
validator_metrics::inc_counter_vec(
&validator_metrics::SIGNED_BLOCKS_TOTAL,
&[validator_metrics::SAME_DATA],
);
```

```
Err(Error::SameData)
}
Err(NotSafe::UnregisteredValidator(pk)) => {
warn!("Not signing block for unregistered validator {:?}", pk);
validator_metrics::inc_counter_vec(
&validator_metrics::SIGNED_BLOCKS_TOTAL,
&[validator_metrics::UNREGISTERED],
);
Err(Error::UnregisteredValidator(pk))
}
Err(NotSafe::InvalidBlock(reason)) => {
error!("Not signing invalid block: {:?}", reason);
validator_metrics::inc_counter_vec(
&validator_metrics::SIGNED_BLOCKS_TOTAL,
&[validator_metrics::INVALID],
);
Err(Error::InvalidBlock(reason))
}
Err(NotSafe::DoubleVote(existing_block)) => {
error!("Not signing block due to double vote: {:?}", existing_block);
validator_metrics::inc_counter_vec(
&validator_metrics::SIGNED_BLOCKS_TOTAL,
&[validator_metrics::DOUBLE_VOTE],
```

```
);
Err(Error::DoubleVote(existing_block))
}
}
}
```

서명 보안 검사:

1. **슬롯 검증**: 미래 슬롯의 블록은 서명하지 않음
2. **Doppelganger 보호**: 중복 검증자 감지
3. **슬래싱 보호**: 이중 투표 방지
4. **서명 실행**: 모든 검사를 통과한 경우에만 서명

블록 제출 과정

1. 서명된 블록 제출 (publish_signed_block_contents)

```
async fn publish_signed_block_contents(
&self,
signed_block: &SignedBlock<S::E>,
beacon_node: BeaconNodeHttpClient,
) -> Result<(), BlockError> {
match signed_block {
SignedBlock::Full(signed_block) => {
let _post_timer = validator_metrics::start_timer_vec(
```

```
&validator_metrics::BLOCK_SERVICE_TIMES,
&[validator_metrics::BEACON_BLOCK_HTTP_POST],
);
beacon_node
.post_beacon_blocks_v2_ssz(signed_block, None)
.await
.or_else(|e| {
handle_block_post_error(e, signed_block.signed_block().message().slot())
})?
}
SignedBlock::Blinded(signed_block) => {
let  _post_timer = validator_metrics::start_timer_vec(
&validator_metrics::BLOCK_SERVICE_TIMES,
&[validator_metrics::BLINDED_BEACON_BLOCK_HTTP_POST],
);
beacon_node
.post_beacon_blinded_blocks_v2_ssz(signed_block, None)
.await
.or_else(|e| handle_block_post_error(e, signed_block.message().slot()))?
}
}
Ok::<_, BlockError>(())
```

```
}
```

2. 에러 처리 (handle_block_post_error)

```
fn handle_block_post_error(err: eth2::Error, slot: Slot) -> Result<(), BlockErr

// Handle non-200 success codes.

if let Some(status) = err.status() {

if status == StatusCode::ACCEPTED {

info!(

%slot,

status_code = status.as_u16(),

"Block is already known to BN or might be invalid"

);

return Ok(());

} else if status.is_success() {

debug!(

%slot,

status_code = status.as_u16(),

"Block published with non-standard success code"

);

return Ok(());

}

}
```

```
Err(BlockError::Irrecoverable(format!(
"Error from beacon node when publishing block: {err:?}",
)))
}
```

ProposerFallback 메커니즘

구조체 정의

```
pub struct ProposerFallback<T> {
beacon_nodes: Arc<BeaconNodeFallback<T>>,
proposer_nodes: Option<Arc<BeaconNodeFallback<T>>>,
}
```

Fallback 전략

1. Proposer 우선 요청 (request_proposers_first)

```
pub async fn request_proposers_first<F, Err, R>(&self, func: F) -> Result<(),
where
F: Fn(BeaconNodeHttpClient) -> R + Clone,
R: Future<Output = Result<(), Err>>,
Err: Debug,
{
```

```
// 제안자 노드가 있으면 먼저 시도
if let Some(proposer_nodes) = &self.proposer_nodes {
if proposer_nodes
.request(ApiTopic::Blocks, func.clone())
.await
.is_ok()
{
return Ok(());
}
}

// 제안자 노드 실패 시 일반 노드 사용
self.beacon_nodes.request(ApiTopic::Blocks, func).await
}
```

2. Proposer 후순위 요청 (request_proposers_last)

```
pub async fn request_proposers_last<F, O, Err, R>(&self, func: F) -> Result<O
where
F: Fn(BeaconNodeHttpClient) -> R + Clone,
R: Future<Output = Result<O, Err>>,
Err: Debug,
{
```

```
// 일반 노드 먼저 시도
let beacon_nodes_result = self.beacon_nodes.first_success(func.clone()).await;


match (beacon_nodes_result, &self.proposer_nodes) {
(Ok(success), _) => Ok(success),
(Err(e), None) => Err(e),
// 일반 노드 실패 시 제안자 노드 시도
(Err(_), Some(proposer_nodes)) => proposer_nodes.first_success(func).await,
}
}
```

Fallback 전략:

- **블록 요청**: 일반 노드 우선 (네트워크 상태 정보가 더 좋음)
- **블록 제출**: 제안자 노드 우선 (DoS 공격 보호)

성능 모니터링 및 메트릭

주요 메트릭

```
// validator_client/validator_metrics/src/lib.rs
pub static BLOCK_SERVICE_TIMES: LazyLock<Result<HistogramVec>> = LazyLock::new(
try_create_histogram_vec(
"vc_beacon_block_service_task_times_seconds",
"Duration to perform beacon block service tasks",
```

```
&["task"],
)
});

pub static BLOCK_SIGNING_TIMES: LazyLock<Result<Histogram>> = LazyLock::new(||
try_create_histogram(
"vc_beacon_block_signing_times_seconds",
"Duration to sign a beacon block",
)
});

pub static SIGNED_BLOCKS_TOTAL: LazyLock<Result<IntCounterVec>> = LazyLock::new
try_create_int_counter_vec(
"vc_signed_beacon_blocks_total",
"Total count of attempted block signings",
&["status"],
)
});
```

메트릭 수집 지점

- `FULL_UPDATE` : 전체 블록 제안 과정
- `BEACON_BLOCK` : 개별 블록 처리
- `BEACON_BLOCK_HTTP_GET` : 미서명 블록 요청

- `BEACON_BLOCK_HTTP_POST` : 서명된 블록 제출
- `BLINDED_BEACON_BLOCK_HTTP_POST` : 블라인드 블록 제출

에러 처리 및 복구

BlockError 타입

```
#[derive(Debug)]
pub  enum BlockError {
/// 복구 가능한 에러 (재시도 가능)
Recoverable(String),
/// 복구 불가능한 에러 (이미 서명된 경우)
Irrecoverable(String),
}
```

에러 분류 전략

- **Recoverable**: 네트워크 오류, 일시적 beacon node 문제
- **Irrecoverable**: 서명 후 제출 실패, 중복 서명 시도

보안 메커니즘

1. 슬래싱 보호

- 이중 투표 방지
- 슬래싱 데이터베이스를 통한 이력 관리

- 동일한 슬롯에 대한 중복 서명 방지

2. Doppelganger 보호

- 네트워크상의 중복 검증자 감지
- 안전이 확인될 때까지 서명 지연
- 검증자 활성화 상태 모니터링

3. 시간 기반 보호

- 미래 슬롯 블록 서명 방지
- 만료된 슬롯 처리 건너뛰기
- 제네시스 슬롯 특별 처리

결론

Block Service는 Lighthouse Validator Client의 핵심 컴포넌트로서 안전하고 효율적인 블록 제안 기능을 제공합니다. 다층적인 보안 메커니즘과 fallback 전략을 통해 높은 가용성을 보장하며, 상세한 메트릭 수집을 통해 성능 모니터링이 가능합니다.

주요 특징:

- **안전성**: 슬래싱 보호 및 doppelganger 보호
- **가용성**: 다중 beacon node fallback 지원
- **성능**: 병렬 처리 및 상세한 메트릭 수집
- **유연성**: Full 블록과 Blinded 블록 모두 지원

이러한 설계를 통해 Block Service는 Ethereum 2.0 네트워크에서 안정적이고 효율적인 블록 제안 서비스를 제공합니다.